

PZ-CH9121 以太网转串口模块开发手册

本手册将教大家如何在普中 F103 最小系统&玄武&朱雀开发板上使用 PZ-CH9121 以太网转串口模块。本章分为如下几部分内容：

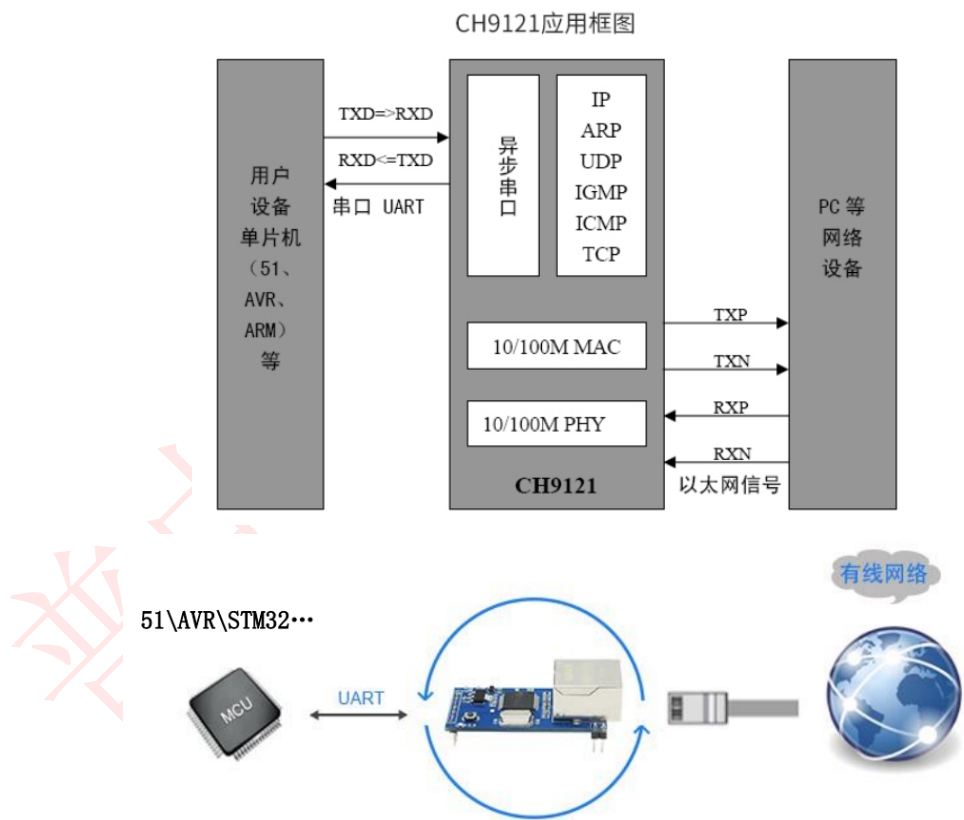
- 1 PZ-CH9121 以太网转串口模块简介
- 2 模块使用说明
- 3 硬件设计
- 4 软件设计
- 5 实验现象

1 PZ-CH9121 以太网转串口模块简介

1.1 模块简介

PZ-CH9121 模块是深圳普中科技推出的一款以太网转串口数据透传模块。模块内部集成 TCP/IP 协议栈，可实现网络数据包和串口数据的双向透明传输；模块主控自带 10/100M 以太网介质传输层（MAC）和物理层（PHY），完全兼容 IEEE802.3 协议，有 TCP Client、TCP Server、UDP Client 、UDP Server 4 种工作模式，串口波特率最高可支持到 921600bps，可通过上位机软件或者串口命令可对模块工作模式、波特率、IP 地址等参数进行快速配置。

模块一般应用框架如下：



1.2 模块参数

PZ-CH9121 以太网转串口模块详细参数如下图：

PZ-CH9121以太网转串口模块参数	
模块功能特点	1、双供电接口：5V 或 3.3V，任选其一
	2、内部自带以太网介质传输层（MAC）和物理层（PHY）
	3、支持 10/100M，全双工/半双工自适应以太网接口，兼容 802.3 协议
	4、Auto-MDI/MIDX 功能，交叉直连网线任意连接，自动切换
	5、支持 DHCP 自动获取 IP 地址，支持 DNS 域名访问
	6、可以通过 UDP 广播协议查询网络内的所有设备
	7、工作模式支持 TCP Client、TCP Server 和 UDP Client、UDP Server 4 种模式
	8、RJ45 状态指示灯，RJ45 接口内置隔离变压器，2KV 隔离。
	9、串口支持全双工和半双工串口通讯，支持 RS485 收发自动切换
	10、串口波特率支持 300bps ~ 921600bps
	11、功耗：工作电流@120--200mA
	12、串口TTL电平，引脚IO兼容3.3V和5V电压
模块软件特点	1、支持串口命令配置
	2、支持通过上位机软件模块的网络、串口参数
	3、配置参数断电数据保存
PCB尺寸及接口	1、层数：2 层
	2、尺寸：56.77*29.72 mm
	3、接口：RJ45 以太网接口&2.54mm 标准排针接口

1.3 模块默认出厂设置

PZ-CH9121 以太网串口透传模块出厂默认工作在 TCP Client 模式，网络相关各参数默认配置如下：

网络参数	(1) 设备 IP: 192.168.1.200
	(2) 子网掩码: 255.255.255.0
	(3) 默认网关: 192.168.1.1
	(4) 模块端口: 2000
	(5) 目的 IP: 192.168.1.100
	(6) 目的端口: 1000
	(7) 重连次数: 无限次
串口参数	(1) 波特率: 9600
	(2) 超时: 0
	(3) 数据位: 8 ; 停止位 : 1 ; 校验 : 无
	(4) 清空串口缓冲区: 从不清空

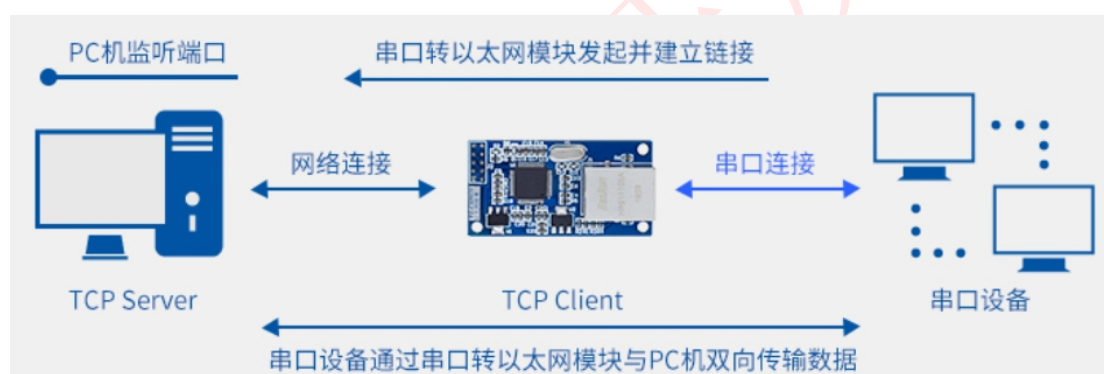
1.4 模块配置及工作模式说明

模块参数配置分为基础设置和端口配置两个部分，基础配置主要包括，设备

名网络参数，串口协商认证功能。串口协商认证功能默认处于关闭状态，默认通过硬件 CFG 引脚进入串口设置模式，开启后可以通过串口协商的方式进入串口设置模式。模块支持 DHCP 和手动两种方式设置网络基础参数。

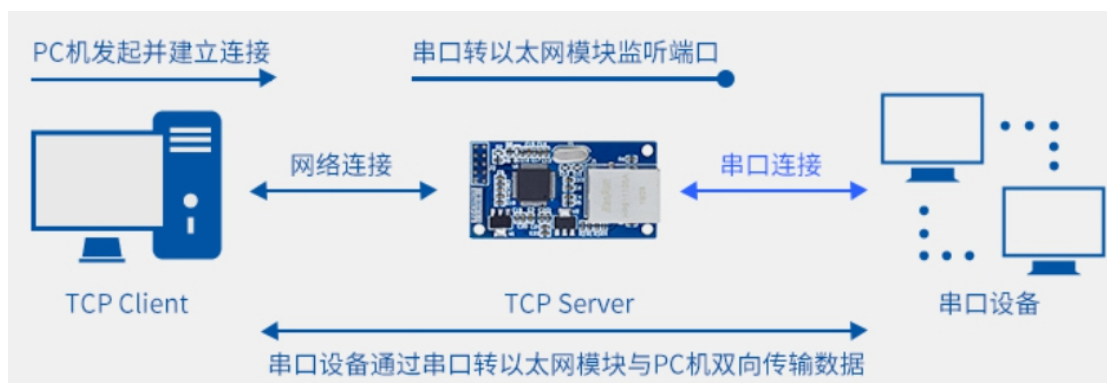
1.4.1 TCP Client 模式

在 TCP Client 模式，模块上电后，会主动连接 TCP Server 端，连接建立后，可实现网络数据和串口数据的双向透明传输。此模式下，TCP Server 的 IP 需对模块可见，可见的含义是指通过模块所在的 IP 可直接 PING 通服务器 IP。TCP Client 模式下，支持本地端口随机，支持通过域名访问远端服务区，芯片内部默认开启 TCP 底层 Keep Alive 保活机制，可以检测出设备掉线。TCP 客户端应用模型如下，适合于现场数据采集，上传服务器模式。



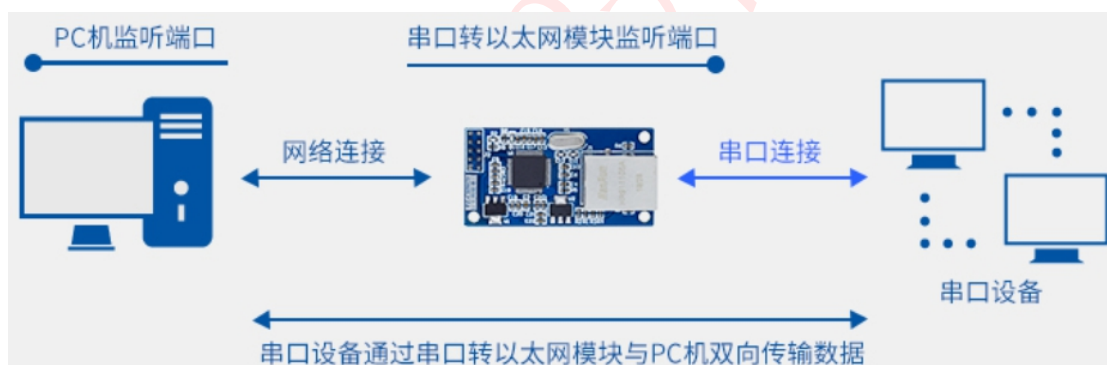
1.4.2 TCP Server 模式

在 TCP Server 模式，模块上电后，会监听本地端口是否有客户端请求连接，连接建立后，可实现网络数据和串口数据的双向透明传输。此模式下，TCP Client 的 IP 需对模块可见，可见的含义是指通过客户端 IP 可直接 PING 通模块 IP。模块需要配置的网络参数有：工作模式、设备 IP、子网掩码、默认网关、设备端口。而目的 IP、目的端口、此模式下，同时只能支持一条 TCP 客户端连接。



1.4.3 UDP Client 模式

在 UDP Client 模式，模块上电后，会把发往本地端口的数据（来自于目的 IP 和端口）透明转发到模块串口，同理，发往模块串口的数据也会通过 UDP 方式转发至设定的目的 IP 和端口。此模式下，模块需要配置的网络参数有：工作模式、设备 IP、子网掩码、默认网关、设备端口、目的 IP。



1.4.4 UDP Server 模式

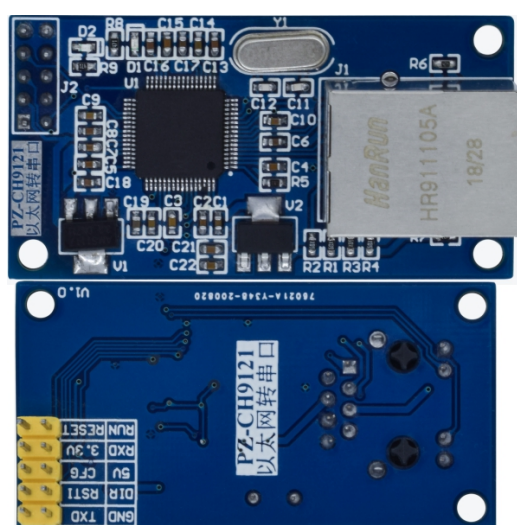
在 UDP Server 模式，接收发往本地 IP 和端口的所有数据并转发至串口，发往模块串口的数据也会通过 UDP 方式转发至与之通信的 UDP 的 IP 和端口。此模式下，模块需要配置的网络参数有：工作模式、设备 IP、子网掩码、默认网关、设备端口。

2 模块使用说明

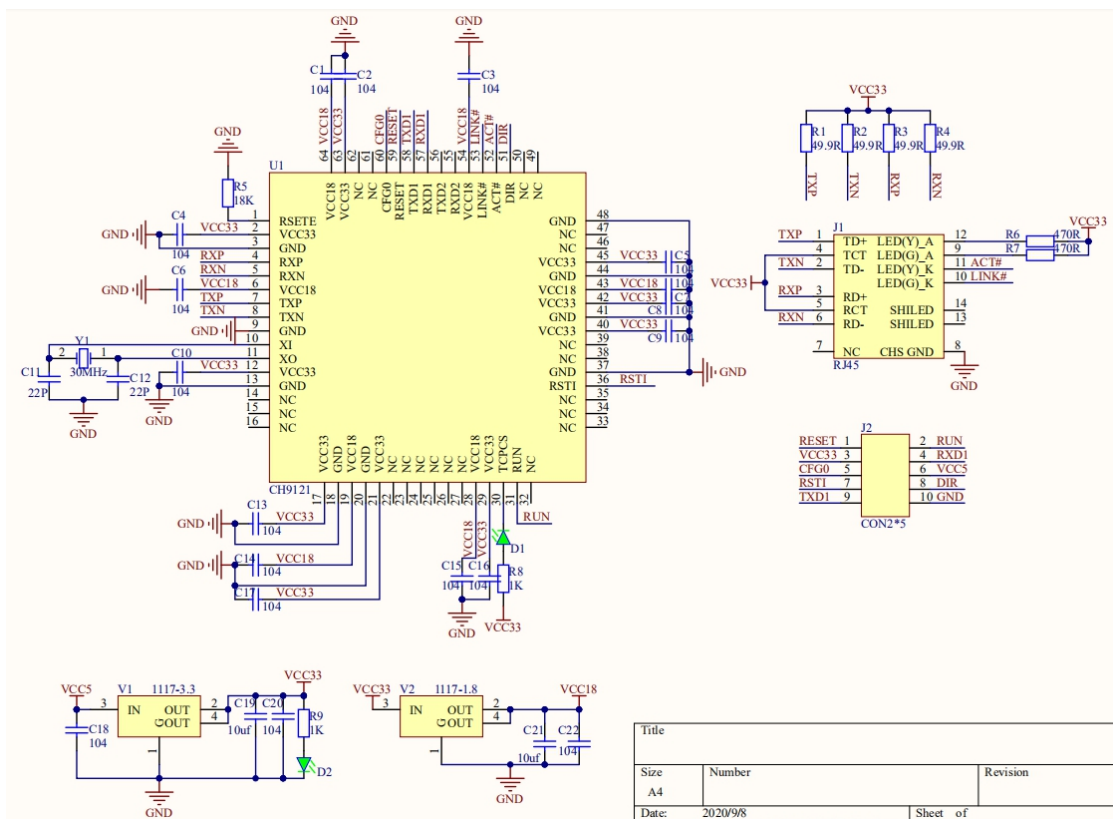
2.1 模块引脚说明

模块通过 2*5 的排针（2.54mm 间距）同外部连接。模块可以使用杜邦线与普中 F103 最小系统&玄武&朱雀等开发板连接进行测试。对应开发板都提供有相应例程，用户可以直接在这些开发板上，对模块进行测试。

PZ-CH9121 模块外观如下图所示：



PZ-CH9121 模块原理图如下图所示：



从上图可以看出,通过 J2 排针的 VCC5 或 VCC33 输入,经过 V1/V2 稳压芯片,转换为 3.3V 和 1.8V 供 CH9121 芯片电源。

J2 接口的管脚之所以这样分配,主要是为了配合普中带 WIFI 接口的板子,无需连线,直接插入板载 WIFI 接口即可使用。对于普中不含 WIFI 接口的板子就需要手动连线,后面小节会介绍如何接线。

模块通过一个 2*5 的排针 (J2) 同外部电路连接,各引脚的详细描述如下图所示:

引脚	I/O	功能及使用说明
GND		外部控制器需要和模块的 GND 共地
TXD	TX	模块串口发送端 UART_TX, 连接外部控制器串口的 RX 端
DIR	I	暂时不使用,可悬空
RSTI	I	外部复位输入,低电平有效,内置上拉电阻,默认为高
5V		外部供电引脚,3.3V 或 5V 供电可选其一
CFG	I	串口配置模式设置脚,内置上拉,检测到低电平时,进入串口配置模式;高电平退出串口配置模式
RXD	RX	模块串口接收端 UART_RX, 连接外部控制器串口的 TX 端
3.3V		外部供电引脚,3.3V 或 5V 供电可选其一
RUN	I	模块固件更新使能引脚,内置上拉,检测到低电平时,进入固件更新模式,高电平退出模式
RESET	I	恢复出厂设置,芯片上电检测,低电平有效(保持 1s 即可),默认为高

注意: 模块正常工作不需要复位或者串口配置模块时, RSTI、CFG 引脚可悬空。

2.2 使用上位机配置模块

PZ-CH9121 以太网转串口模块使用非常简单，支持上位机可视化快速配置，也可以通过指令对模块的网络、串口参数进行设置，所有参数掉电保存。

要使用上位机对模块进行配置，即要通过模块串口与电脑 USB 口进行连接，可以使用一个 USB 转 TTL 模块，如果使用普中 STM32 开发板的用户，可直接使用开发板板载的 USB 转 TTL 模块 P4 端子，如下：



按照如下接线方式将模块与开发板连接：（PZ-CH9121 模块-->STM32 开发板）

GND-->GND

TXD-->URXD (P4 端子)

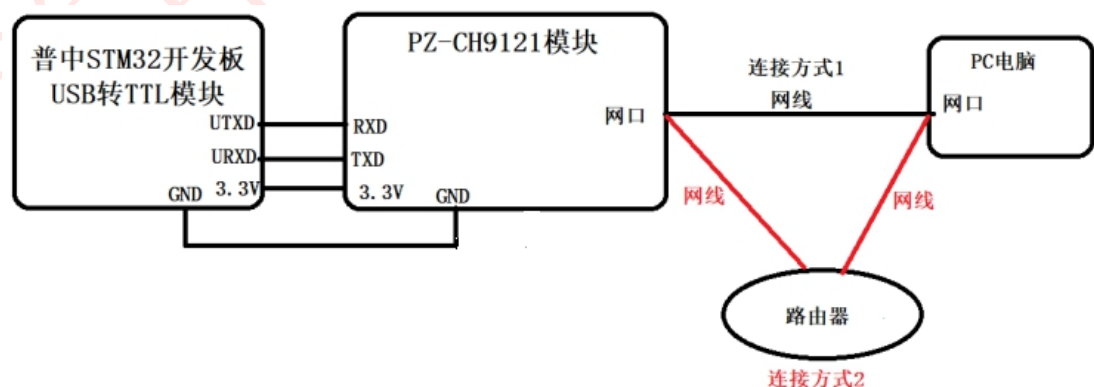
RXD-->UTXD (P4 端子)

3.3V-->3.3V

还需使用一根网线将模块的网口与电脑网口或与电脑连接的同一路由器网口。

注意要保证模块与电脑处于同一局域网内。

连接结构示意图如下所示：



2.2.1 TCP Client 模式配置

(1) 上述硬件连接好后，打开电源，模块供电正常，打开“..\3—软件工具\NetModuleConfig_开放 WIFI.exe”软件，点击“搜索设备”，硬件连接正常时设备列表内会显示 CH9121 模块设备，如下所示：



此时，鼠标双击设备列表中显示的 CH9121 设备，可获取模块的 IP 地址、子网掩码、网关等信息。如下所示：



(2) 若需要对模块进行配置，可在右窗口进行参数的设置，比如：我的电

脑本地 IP 地址设置如下：

Internet 协议版本 4 (TCP/IPv4) 属性

常规

如果网络支持此功能，则可以获取自动指派的 IP 设置。否则，你需要从网络系统管理员处获得适当的 IP 设置。

☐ 自动获得 IP 地址(O)

☒ 使用下面的 IP 地址(S):

IP 地址(I): 192 . 168 . 1 . 106

子网掩码(U): 255 . 255 . 255 . 0

默认网关(D): 192 . 168 . 1 . 1

☐ 自动获得 DNS 服务器地址(B)

☒ 使用下面的 DNS 服务器地址(E):

首选 DNS 服务器(P): 192 . 168 . 1 . 1

备用 DNS 服务器(A):

☐ 退出时验证设置(L)

高级(V)...

确定 取消

那么就需要在下图配置界面中将“目的 IP”设置为 192. 168. 1. 106，目的端口可以任意，这里保持默认的 8800 值。然后串口波特率等参数也保持默认，即波特率 9600、数据位 8、停止位 1、无校验。当然你也可以根据自己的需求修改波特率等参数。**因为模块默认即为 TCP Client 模式，所以网络模式自动为 TCP CLIENT**，如下所示：

网络配置工具

适配器: 1. Realtek PCIe GbE Family Cont... 刷新网卡

设备列表 (双击设备列表中的模块，可以获取对应设备的配置)

设备名	设备IP	设备MAC	版本
CH9121	192.168.1.200	84:C2:E4:E8:45:55	37

搜索设备

恢复出厂设置 加载配置文件 保存配置文件

基础设置

设备名: CH9121 (?)

DHCP: ☐ 开启 (?)

设备IP: 192 . 168 . 1 . 200 (?)

子网掩码: 255 . 255 . 255 . 0 (?)

网关: 192 . 168 . 1 . 1 (?)

串口协商配置: ☐ 开启 (?)

操作状态: 获取配置成功

启用端口2

端口1

网络模式: TCP CLIENT (?)

本地端口: ☐ 随机 8800 (?)

目的IP/域名: IP (?)

目的IP: 192 . 168 . 1 . 106 (?)

目的端口号: 8800 (?)

串口波特率: 9600 (?)

串口数据位: 8 (?)

串口停止位: 1 (?)

串口校验位: 无校验 (?)

网线断开: ☒ 关闭网络连接 (?)

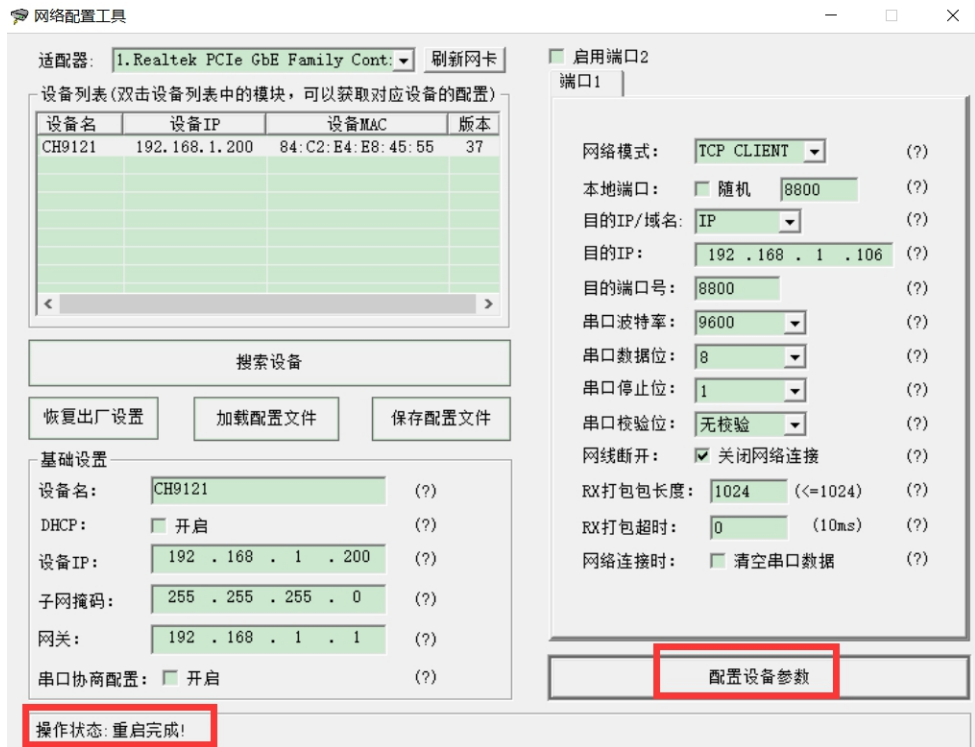
RX打包长度: 1024 (<=1024) (?)

RX打包超时: 0 (10ms) (?)

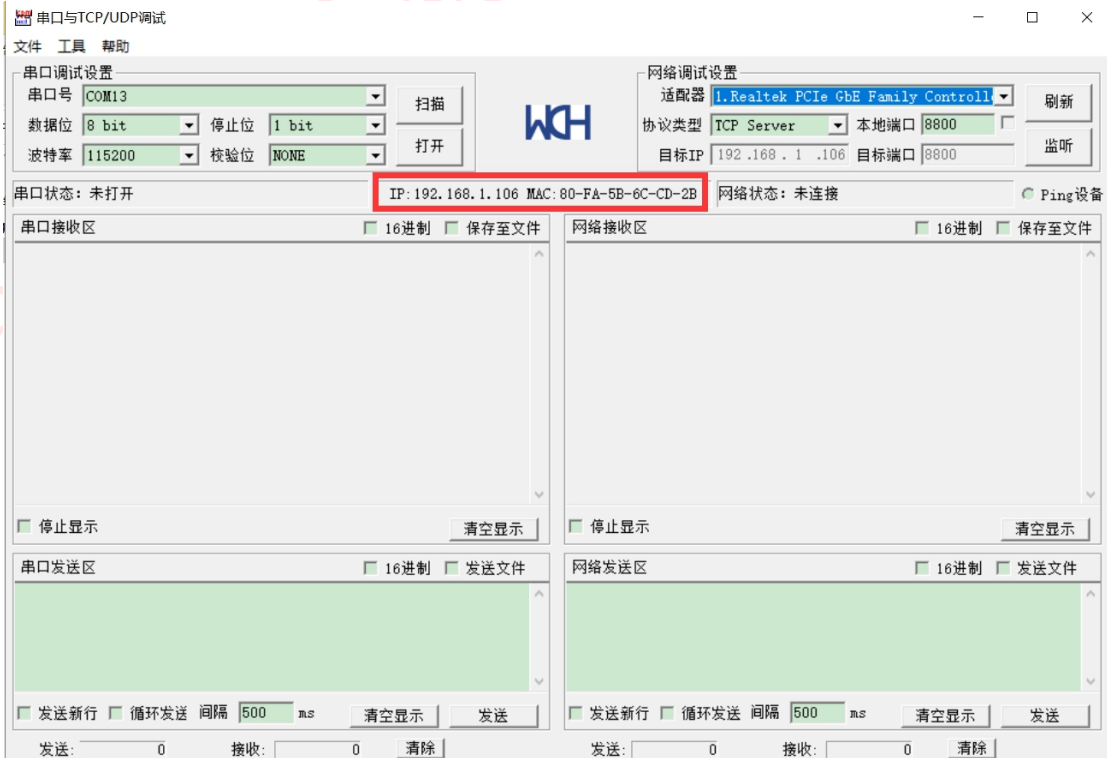
网络连接时: ☐ 清空串口数据 (?)

配置设备参数

设置好后，点击“配置设备参数”，开始将参数传输到模块中存储。配置过程中左下角状态栏会显示，配置完成后会显示重启完成，如下所示：



(3) 此时，我们验证下 TCP CLIENT 模式是否配置成功，打开“.. \3—软件工具\SER-NET.exe”软件，会自动识别电脑本地 IP 地址，如下所示：



下面我们选择模块连接开发板 USB 转 TTL 模块的串口号，设置对应的波特率

等参数，以及网络协议类型，**因为模块使用 TCP 客户端，所以电脑即为 TCP 服务器（TCP Server）**，本地端口、目的 IP 地址和目的端口设置即为前面配置参数所设置的值。然后打开串口，打开监听，若提示连接成功，参数设置及硬件连接都是 OK 的。如下所示：



(4) 下面即可验证模块 TCP 客户端与电脑 TCP 服务器通信，①在串口发送区发送字符，网络接收区即会显示，说明我们向模块串口发送数据后，通过模块网络接口传输到电脑服务器端了。②在网络发送区发送字符，串口接收区即会显示，说明我们从电脑网口发送数据后，通过模块网口接收转换为串口传输。显示效果如下：

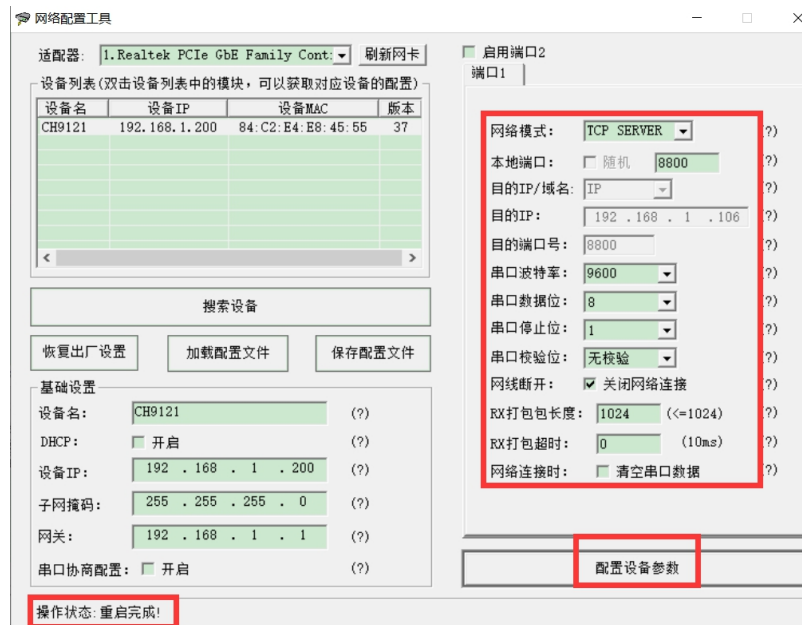


至此，PZ-CH9121 模块的 TCP CLIENT 模式的配置就介绍完毕。

2.2.2 TCP Server 模式配置

前面介绍了 TCP Client 模式的配置，软件的操作我们就不介绍了，重点看下如何配置。

(1) 打开“NetModuleConfig_开放 WIFI.exe”软件，将“网络模式”选择为 TCP SERVER，其他保持默认，然后点击“配置设备参数”，等待配置完成。如下：



(2) 打开“SER-NET.exe”软件，设置好串口波特率等参数，将协议类型设置为 TCP CLIENT，且设置好目的 IP 地址和端口等参数，模块默认的 IP 地址为 192.168.1.200，端口为 8800，打开串口和网络监听，如下所示：



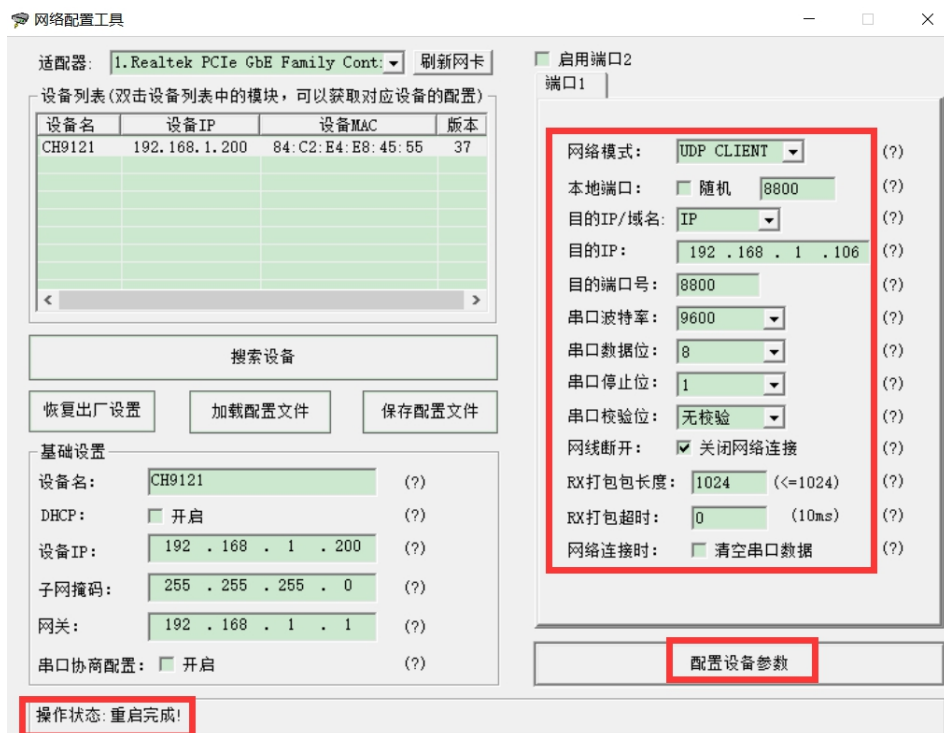
(3) 下面即可验证模块 TCP 服务器与电脑 TCP 客户端通信，①在串口发送区发送字符，网络接收区即会显示。②在网络发送区发送字符，串口接收区即会显示。显示效果如下：



2.2.3 UDP Client 模式配置

前面介绍了 TCP 模式的配置，软件的操作我们就不介绍了，重点看下如何配置。

(1) 打开“NetModuleConfig_开放 WIFI.exe”软件，将“网络模式”选择为 TCP SERVER，其他保持默认，然后点击“配置设备参数”，等待配置完成。如下：



(2) 打开“SER-NET. exe”软件，设置好串口波特率等参数，将协议类型设置为 UDP，且设置好目的 IP 地址和端口等参数，模块默认的 IP 地址为 192.168.1.200，端口为 8800，打开串口和网络监听，如下所示：



(3) 下面即可验证模块 UDP 通信，①在串口发送区发送字符，网络接收区即会显示。②在网络发送区发送字符，串口接收区即会显示。显示效果如下：



2.2.3 UDP Server 模式配置

该模式配置与 UDP Client 模式配置一致，这里就不重复。

2.3 使用串口指令配置模块

进入与退出串口指令配置的方法是硬件设置 CFG 引脚, 当模块主控检测到 CFG 引脚 CHG 引脚为低电平时, 模块接收的串口数据则认为是配置数据。当 CFG 引脚拉高则退出配置模式。所以该方式的配置与前面上位机配置仅多了对模块 CFG 拉低的操作。

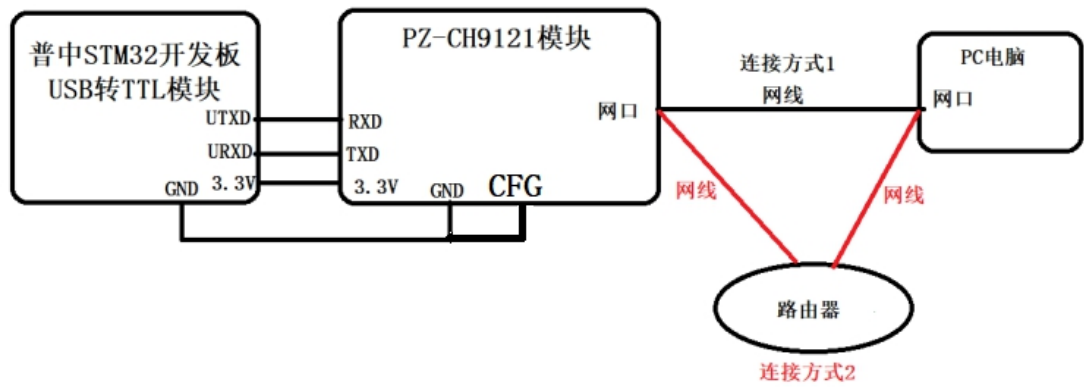
按照如下接线方式将模块与开发板连接: (PZ-CH9121 模块-->STM32 开发板)

GND-->GND
TXD-->URXD (P4 端子)
RXD-->UTXD (P4 端子)
CFG-->GND (进入串口配置)
3.3V-->3.3V

还需使用一根网线将模块的网口与电脑网口或与电脑连接的同一路由器网

口。注意要保证模块与电脑处于同一局域网内。

连接结构示意图如下所示：



模块串口配置命令格式为：0x57 0xAB 命令码 数据。

2.3.1 模块写命令操作

命令码	功能	举例(注释)
写命令流程：发送命令码 1(0x57 0xab + 命令码 1+数据)+ 等待 CH9121 ACK(0xAA)+发送命令码 2(0x57 0xab + 命令码 2+数据)+ 等待 CH9121 ACK(0xAA)+.....+发送命令码 n(0x57 0xab + 命令码 n+数据)+ 等待 CH9121 ACK(0xAA)+发送数据更新命令 (0x57 0xab 0x0d)+ 等待 CH9121 ACK(0xAA)+发送执行命令 (0x57 0xab 0x0e)+等待 CH9121 ACK(0xAA)。		
0x0e	命令执行	0x57 0xab 0x0e (命令执行)
0x5e	退出串口配置模式	0x57 0xab 0x5e (退出串口配置模式)
0x02	复位命令，芯片重新运行	0x57 0xab 0x02 (芯片复位)
0x11	设置芯片	IP 0x57 0xab 0x11 0xc0 0xa8 0x01 0xc8(192.168.1.200)
0x12	设置芯片掩码	0x57 0xab 0x12 0xff 0xff 0xff 0x00 (255.255.255.0)
0x13	设置芯片网关	0x57 0xab 0x13 0xc0 0xa8 0x01 0x01(192.168.1.1)
0x10	设置模式： 00:TCP 服务器 01:TCP 客户端 02:UDP 服务器 03:UDP 客户端	0x57 0xab 0x10 0x01 (设置芯片工作在 TCP 客户端模式)
0x14	设置模块本地端口	0x57 0xab 0x14 0xd0 0x07(端口号为 2000，注意十六进制高位在后，低位在前，0x7d0)
0x15	设置模块目的 IP	0x57 0xab 0x15 0xc0 0xa8 0x01 0x64 (192.168.1.100)
0x16	设置芯片目的端口	0x57 0xab 0x16 0xe8 0x03(端口号为 1000，注意十六进制高位在后，低位在前，0x3e8)
0x21	设置串口波特率	0x57 0xab 0x21 0x80 0x25 0x00 0x00 (9600，注意十六进制高位在后，低位在前 0x00002580)
0x22	设置串口校验位数据位停止位：校验： 00: 偶 01: 奇 02: mark 03: Space 04: 无	0x57 0xab 0x22 0x01 0x04 0x08 (1stop，无校验，8data)

应用举例：

通过串口（无论是 MCU 还是 PC）对模块进行配置，需要按照写命令的流程进行操作，每发一条命令，模块如果执行成功都会返回校验值。

设置说明：“→” 串口设备发送；“←” ETH-01 模块返回

(说明：为了方便观看，我们把要输入的“空格”换成“，”，实际按照空格输入)

```
→0x57, 0xAB, 0x10, 0x02 //02: 模块设置成 UDP 广播模式。
←0xAA
→0x57, 0xAB, 0x11, 0xC0, 0xA8, 0x01, 0x0A //本地 IP: 192.168.1.10
←0xAA
→0x57, 0xAB, 0x12, 0xFF, 0xFF, 0xFF, 0x00 //子网掩码: 255.255.255.0
←0xAA
→0x57, 0xAB, 0x13, 0xC0, 0xA8, 0x01, 0x01 //网关: 192.168.1.1
←0xAA
→0x57, 0xAB, 0x14, 0x88, 0x13 //本地端口: 0x1388(5000)
←0xAA
→0x57, 0xAB, 0x15, 0xFF, 0xFF, 0xFF, 0xFF //目的 IP 地址: 255.255.255.255
←0xAA
→0x57, 0xAB, 0x16, 0x70, 0x17 //目的端口: 0x1770(6000)
←0xAA
→0x57, 0xAB, 0x21, 0x00, 0xC2, 0x01, 0x00 //串口波特率: 0x0001c200(1152000)
←0xAA
→0x57, 0xAB, 0x0D //更新配置参数至 EEPROM
←0xAA
→0x57, 0xAB, 0x0E //执行配置
←0xAA
```

2.3.2 模块读命令操作

命令码	功能	串口返回值举例说明（注释）
读命令,获取配置 (0x57 0xab+命令码, 比如发送: 0x57 0xab 0x60 查询芯片工作模式, 模块串口返回: 0x01, 说明模块当前工作在 TCP 客户端下)		
0x60	读取芯片工作模式, 返回 1 字节	0x00:(模块工作于 TCP 服务器)
		0x01:(模块工作于 TCP 客户端)
		0x02:(模块工作于 UDP 服务器)
		0x03:(模块工作于 UDP 客户端)
0x61	读取芯片 IP 地址, 返回 4 字节	0xC0 0xA8 0x01 0x6F: (模块的 IP 地址为: 192.168.1.111)
0x62	读取芯片掩码, 返回 4 字节	0xFF 0xFF 0xFF 0x00: (模块的芯片掩码为: 225.225.225.0)
0x63	读取芯片网关, 返回 4 字节	0xC0 0xA8 0x01 0x01: (模块的网关地址: 192.168.1.1)
0x64	读取芯片本地端口号, 返回 2 字节	0xD0 0x07(模块本地端口: 2000, 注意十六进制高位在后, 地位在前 0x70d)
0x65	读取芯片目的 IP 地址, 返回 4 字节	0xC0 0xA8 0x01 0x72: (模块的目的 IP 地址: 192.168.1.114)
0x66	读取芯片目的端口号, 返回 2 字节	0x90 0x1F:(模块的目的端口号为: 8080, 注意十六进制, 高位在后, 低位在前, 0x1F90)
0x67	读取 TCP 重试次数, 返回 1 字节	01: (1 次)
0x71	读取串口波特率, 返回 4 字节	0x80 0x25 0x00 0x00(波特率: 9600, 注意十六进制, 高位在后, 低位在前, 0x00002580)
0x72	读取串口校验位数据位停止位, 返回 3 字节	0x01 0x04 0x08(1stop 位, 无校验, 8data 位) 00: 偶 01: 奇 02: mark 03: Space 04: 无
0x73	读取串口超时时间, 返回 1 字节	0x00(读取串口超时时间为 0)
0x81	读取 MAC 地址, 返回 6 字节	0x84 0xC2 0xE4 0xF6 0x0B 0x34:(MAC 地址为 84 C2 E4 F6 0B 34)
0x03	读取 TCP 连接状态 (TCP Client 模式下), 返回 1 字节, 1: 连接 0: 断开	0x01:(TCP 已建立连接)

这里我们就不一一演示这些指令了, 考虑到简单方便, 我们推荐使用上位机来配置。

注意: 当使用串口指令配置完模块后, 记得要将 CFG 管脚接的 GND 线拔掉, 即退出串口指令配置模式。

2.4 模块固件更新

PZ-CH9121 出厂已经默烧写了最新版本的固件，如果用户在使用过程中由于操作或者使用环境等因素致使模块掉固件，可通过此小节介绍更新模块固件。具体操作如下：

①按照如下接线方式将模块与开发板连接：（PZ-CH9121 模块-->STM32 开发板）

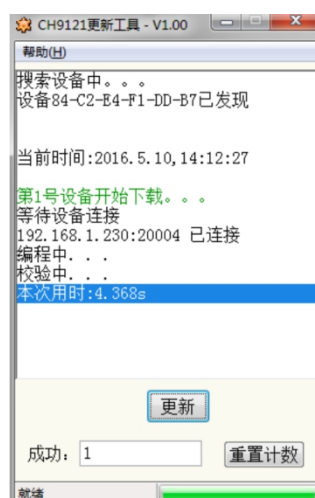
GND-->GND
TXD-->URXD (P4 端子)
RXD-->UTXD (P4 端子)
RUN-->GND（进入固件更新模式）
3.3V-->3.3V

还需使用一根网线将模块的网口与电脑网口或与电脑连接的同一路由器网口。**注意要保证模块与电脑处于同一局域网内。**

②将模块和计算机通过网线接入同一个局域网（注意需要同为 192.168.1.X 网段）。

③打开“..\3--软件工具\ETH-01 模块固件升级工具\WCHISPTool_Setup.exe”软件安装引导。

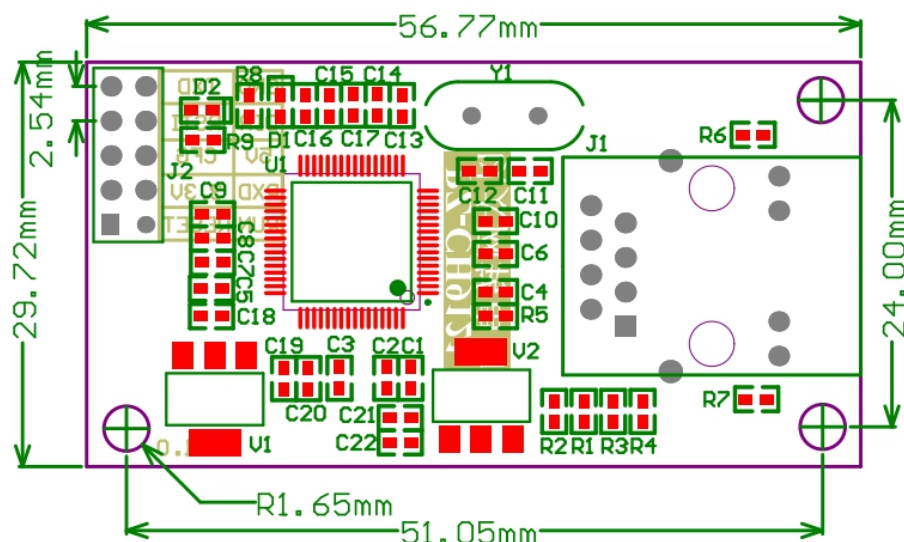
④安装好 WCHISPTool_Setup.exe 软件后，打开“..\3--软件工具\ETH-01 模块固件升级工具\CH9121ISP\CH9121ISPV35.exe”软件，点击更新按钮，工具会自动在局域网内搜索模块并更新固件。如下所示：



更新完成后，将 RUN 脚连接的 GND 线拔下，即退出固件更新模式。

2.5 结构尺寸

PZ-CH9121 模块的尺寸结构如下图所示：



3 硬件设计

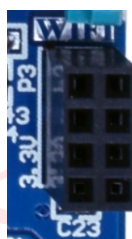
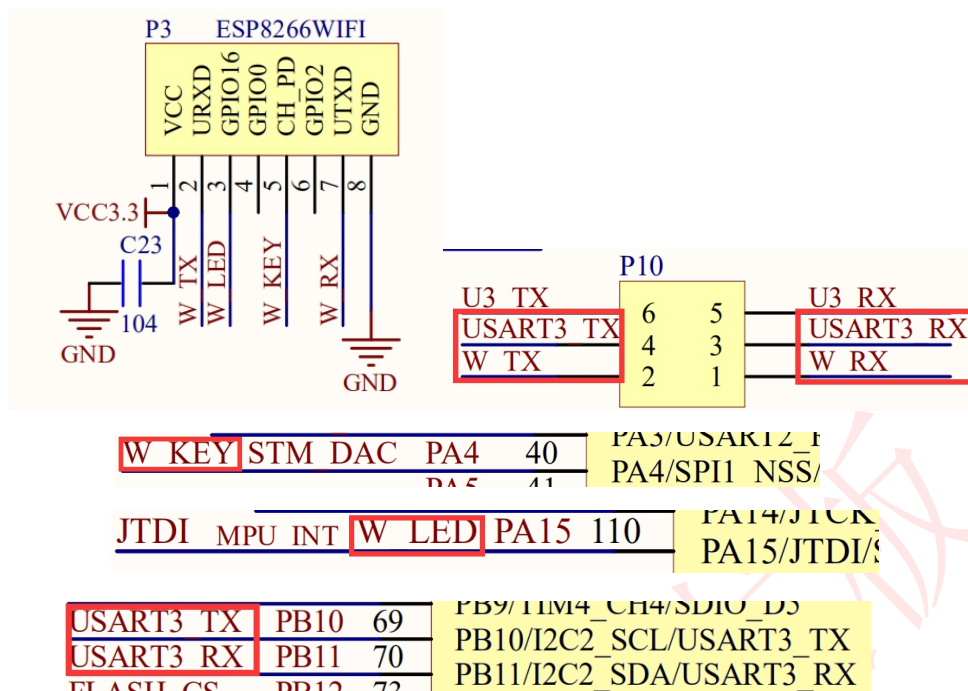
3.1 硬件准备

本实验所需要的硬件资源如下：

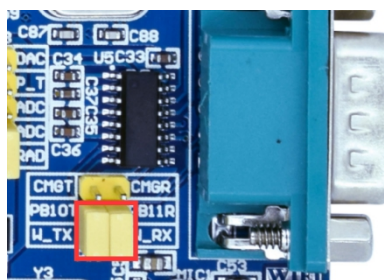
- ①普中 STM32F103 开发板 1 个
- ②PZ-CH9121 模块 1 个
- ③网线 1 条
- ④DS18B20 温度传感器 1 个
- ⑤USB 线 1 条（用于供电和模块与电脑串口调试助手通信）

3.2 模块与开发板连接

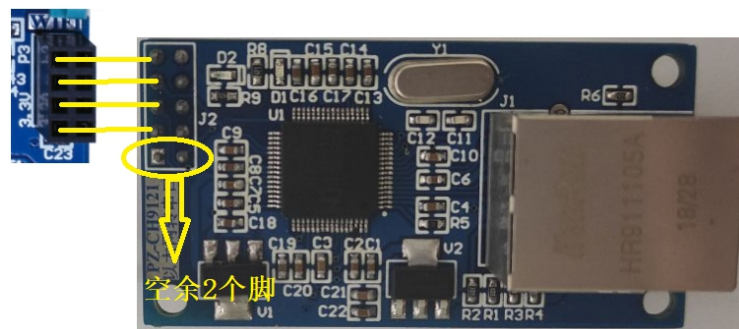
PZ-CH9121 模块可直接与玄武&朱雀开发板板载的 WIFI&蓝牙模块接口（P3）进行连接。该接口与 MCU 连接原理图（以朱雀为例）如下图所示：



从上图看出,WIFI 接口与 STM32 使用时,必须将 P10 的 USART3_TX(PB10 和 W_TX 以及 USART3_RX (PB11) 和 W_RX 连接,才能完成和 STM32 的连接。默认玄武&朱雀的 P10 端子上有黄色跳线帽,直接短接到对应端即可。如下图所示:



注意: PZ-CH9121 模块是 2*5 的接口,而板载 WIFI 是 2*4 的接口,所以会有 2 个管脚插不上,这时可参考模块丝印 3.3V 对应插入,如下所示:



对于 F103 最小系统，是没有 WIFI 接口的，所以需要通过杜邦线将模块管脚与上述 STM32 的 IO 口连接即可。

PZ-CH9121 模块与 STM32 的 IO 口连接关系如下：（PZ-CH9121 模块-->STM32 IO）

```
GND-->GND
TXD-->PB11 (RXD)
RXD-->PB10 (TXD)
3.3V-->3.3V
```

4 软件设计

本实验所实现的功能为：开机检测 DS18B20 温度传感器是否存在，如果检测不成功，则串口输出报错。检测成功之后，串口输出成功，并 DS0 指示灯闪烁提示程序运行正常。结合上位机软件“SER-NET.exe”，即可实现开发板的网络通信，开发板将检测温度值上传网络，可在网络助手控制板载 LED。

本实验代码比较简单，这里不过多介绍，只贴出主要功能代码，ch9121.c 代码如下：

```
1.  #include "ch9121.h"
2.  #include "time.h"
3.
4.  u8 USART3_RX_BUF[USART3_REC_LEN];    //接收缓冲
5.  //通过判断接收连续 2 个字符之间的时间差不大于 10ms 来决定是不是一次连续的数据。
6.  //如果 2 个字符接收间隔超过 10ms,则认为不是 1 次连续数据.也就是超过 10ms 没有接收到
7.  //任何数据,则表示此次接收完毕。
8.  //接收到的数据状态
9.  //[15]:0,没有接收到数据;1,接收到了一批数据。
10. //[14:0]:接收到的数据长度
```

```

11. vu16 USART3_RX_STA=0;          //接收状态标记
12.
13. /*****
14.  * 函 数 名      : CH9121_Init
15.  * 函数功能      : CH9121 初始化函数
16.  * 输    入      : bound:波特率
17.  * 输    出      : 无
18.  *****/
19. void CH9121_Init(u32 bound)
20. {
21.     //GPIO 端口设置
22.     GPIO_InitTypeDef GPIO_InitStructure;
23.     USART_InitTypeDef USART_InitStructure;
24.     NVIC_InitTypeDef NVIC_InitStructure;
25.
26.     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB,ENABLE);
27.     RCC_APB1PeriphClockCmd(RCC_APB1Periph_USART3,ENABLE);
28.     RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO,ENABLE);
29.     RCC_APB2PeriphClockCmd(CFG_PORT_RCC,ENABLE);
30.     RCC_APB2PeriphClockCmd(RSTI_PORT_RCC,ENABLE);
31.
32.     //配置 CFG 管脚
33.     GPIO_InitStructure.GPIO_Pin=CFG_PIN; //选择你要设置的 IO 口
34.     GPIO_InitStructure.GPIO_Mode=GPIO_Mode_Out_PP; //设置推挽输出模式
35.     GPIO_InitStructure.GPIO_Speed=GPIO_Speed_50MHz; //设置传输速率
36.     GPIO_Init(CFG_PORT,&GPIO_InitStructure); /* 初始化 GPIO */
37.     GPIO_SetBits(CFG_PORT,CFG_PIN); //CFG 为高电平，退出串口配置，反之进入串口配置
38.     GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable,ENABLE);
39.
40.     //复位 RSTI 配置
41.     GPIO_InitStructure.GPIO_Pin=RSTI_PIN; //选择你要设置的 IO 口
42.     GPIO_InitStructure.GPIO_Mode=GPIO_Mode_Out_PP; //设置推挽输出模式
43.     GPIO_InitStructure.GPIO_Speed=GPIO_Speed_50MHz; //设置传输速率
44.     GPIO_Init(RSTI_PORT,&GPIO_InitStructure); /* 初始化 GPIO */
45.     GPIO_SetBits(RSTI_PORT,RSTI_PIN);
46.
47.     /* 配置 GPIO 的模式和 IO 口 */
48.     GPIO_InitStructure.GPIO_Pin=GPIO_Pin_10;//TX //串口输出
49.     GPIO_InitStructure.GPIO_Speed=GPIO_Speed_50MHz;
50.     GPIO_InitStructure.GPIO_Mode=GPIO_Mode_AF_PP; //复用推挽输出
51.     GPIO_Init(GPIOB,&GPIO_InitStructure); /* 初始化串口输入 IO */
52.     GPIO_InitStructure.GPIO_Pin=GPIO_Pin_11;//RX //串口输入
53.     GPIO_InitStructure.GPIO_Mode=GPIO_Mode_IN_FLOATING; //模拟输入
54.     GPIO_Init(GPIOB,&GPIO_InitStructure); /* 初始化 GPIO */

```

```

55.
56. //USART3 初始化设置
57. USART_InitStructure.USART_BaudRate = bound; //波特率设置
58. USART_InitStructure.USART_WordLength = USART_WordLength_8b; //字长为 8 位数据格式
59. USART_InitStructure.USART_StopBits = USART_StopBits_1; //一个停止位
60. USART_InitStructure.USART_Parity = USART_Parity_No; //无奇偶校验位
61. USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //无硬件
    数据流控制
62. USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //收发模式
63. USART_Init(USART3, &USART_InitStructure); //初始化串口 2
64.
65. USART_Cmd(USART3, ENABLE); //使能串口 3
66.
67. USART_ClearFlag(USART3, USART_FLAG_TC);
68.
69. USART_ITConfig(USART3, USART_IT_RXNE, ENABLE); //开启相关中断
70.
71. //Usart3 NVIC 配置
72. NVIC_InitStructure.NVIC_IRQChannel = USART3_IRQn; //串口 3 中断通道
73. NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=2; //抢占优先级 2
74. NVIC_InitStructure.NVIC_IRQChannelSubPriority =2; //子优先级 2
75. NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE; //IRQ 通道使能
76. NVIC_Init(&NVIC_InitStructure); //根据指定的参数初始化 VIC 寄存器、
77.
78. TIM7_Int_Init(99,7199); //10ms 中断
79. USART3_RX_STA=0; //清零
80. TIM_Cmd(TIM7,DISABLE); //关闭定时器 7
81. }
82.
83. /*****
84. * 函 数 名 : USART3_IRQHandler
85. * 函数功能 : USART3 中断函数
86. * 输 入 : 无
87. * 输 出 : 无
88. *****/
89.
90. void USART3_IRQHandler(void) //串口 3 中断服务程序
91. {
92.     u8 r;
93.     if(USART_GetITStatus(USART3, USART_IT_RXNE) != RESET) //接收中断
94.     {
95.         r =USART_ReceiveData(USART3); //(USART3->DR); //读取接收到的数据
96.         if((USART3_RX_STA&(1<<15))==0) //接收完的一批数据,还没有被处理,则不再接收其他数据
97.         {

```

```

98.         if(USART3_RX_STA<USART3_REC_LEN) //还可以接收数据
99.         {
100.             TIM_SetCounter(TIM7,0); //计数器清空
101.             if(USART3_RX_STA==0) //使能定时器 7 的中断
102.             {
103.                 TIM_Cmd(TIM7,ENABLE); //使能定时器 7
104.             }
105.             USART3_RX_BUF[USART3_RX_STA++]=r; //记录接收到的值
106.         }
107.         else
108.         {
109.             USART3_RX_STA|=1<<15; //强制标记接收完成
110.         }
111.     }
112. }
113. }
114.
115. void CH9121_USART_Config(FunctionalState NewState)
116. {
117.     if(NewState==ENABLE)
118.         GPIO_ResetBits(CFG_PORT,CFG_PIN);
119.     else
120.         GPIO_SetBits(CFG_PORT,CFG_PIN);
121. }
122.
123. u8 ch9121_cmd_start[]={0x57,0xab,0x0e,'\0'}; //命令执行
124.
125. //CH9121 模式设置: TCP 服务器、TCP 客户端、UDP 服务器、UDP 客户端
126. u8 ch9121_mode_tcp_server[]={0x57,0xab,0x10,0x00,'\0'};
127. u8 ch9121_mode_tcp_clent[]={0x57,0xab,0x10,0x01,'\0'};
128. u8 ch9121_mode_udp_server[]={0x57,0xab,0x10,0x02,'\0'};
129. u8 ch9121_mode_udp_clent[]={0x57,0xab,0x10,0x03,'\0'};
130.
131.
132. //RS232 发送数据
133. //buf: 发送区首地址
134. void CH9121_SendString(u8 *buf)
135. {
136.     while(*buf!='\0') //遇到结束符跳出循环
137.     {
138.         while(USART_GetFlagStatus(USART3, USART_FLAG_TC) == RESET);
139.         USART_SendData(USART3,*buf);
140.         buf++;
141.     }

```

```
142. }
```

我们看下 main.c 代码如下：

```
1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "usart.h"
5.  #include "ch9121.h"
6.  #include <string.h>
7.  #include "ds18b20.h"
8.
9.
10. /*****
11.  * 函数名      : main
12.  * 函数功能    : 主函数
13.  * 输入       : 无
14.  * 输出       : 无
15.  *****/
16. int main()
17. {
18.     u8 i=0;
19.     u8 reclen=0;
20.     u8 res=0;
21.     float temper;
22.     u8 tempbuf[20];
23.
24.     SysTick_Init(72);
25.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
26.     LED_Init();
27.     USART1_Init(115200);
28.     CH9121_Init(9600);
29.     while(DS18B20_Init())
30.     {
31.         printf("DS18B20 检测失败，请插好!\r\n");
32.         delay_ms(500);
33.     }
34.     printf("DS18B20 检测成功!\r\n");
35.
36.     while(1)
37.     {
38.         if(USART3_RX_STA&0X8000)//接收到一次数据
39.         {
40.             reclen=USART3_RX_STA&0X7FFF; //得到数据长度
41.             USART3_RX_BUF[reclen]='\0'; //加入结束符
42.             printf("reclen=%d\r\n",reclen);
```

```

43.         printf("USART3_RX_BUF=%s\r\n",USART3_RX_BUF);
44.         USART3_RX_STA=0;
45.     }
46.
47.     if(strcmp((char *)USART3_RX_BUF,"LED_ON")==0)
48.         LED2=0;
49.     else if(strcmp((char *)USART3_RX_BUF,"LED_OFF")==0)
50.         LED2=1;
51.
52.     i++;
53.     if(i%50==0)
54.     {
55.         LED1=!LED1;
56.         temper=DS18B20_GetTemperture();
57.         if(temper<0)
58.         {
59.             printf("检测的温度为: -");
60.         }
61.         else
62.         {
63.             printf("检测的温度为: ");
64.         }
65.         printf("%.2f°C\r\n",temper);
66.         sprintf((char *)tempbuf,"温度=%.2f°C\r\n",temper);
67.         CH9121_SendString(tempbuf);
68.     }
69.     delay_ms(10);
70. }
71. }

```

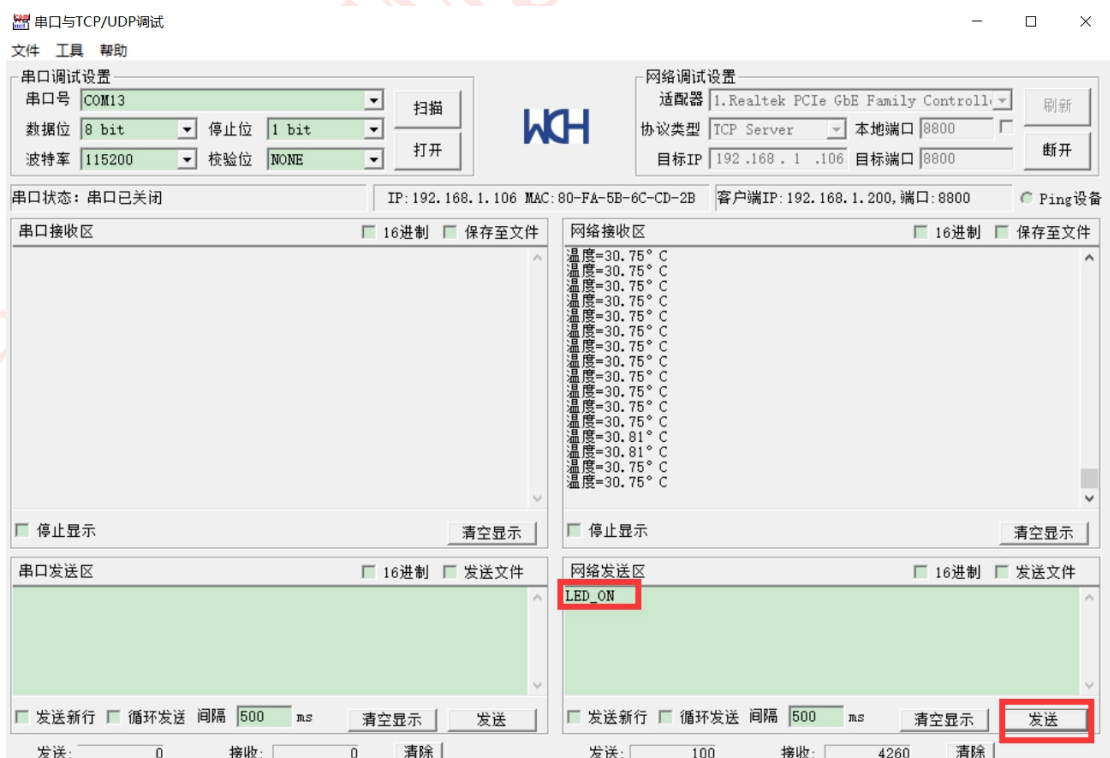
从代码中可以看到，网络端需要发送字符串“LED_ON”才能点亮板载 LED，如果发送“LED_OFF”则关闭 LED。并且开发板间隔 500ms 发送采集温度到网络端。

5 实验现象

首先，请先确保硬件都已经连接好，然后代码编译成功之后，将代码下载到 STM32 开发板上。打开“..[3—软件工具\SER-NET.exe](#)”软件，根据前面对模块工作模式及参数的配置，即可按照模块配置好的参数来设置值，然后打开网络监听，如下：

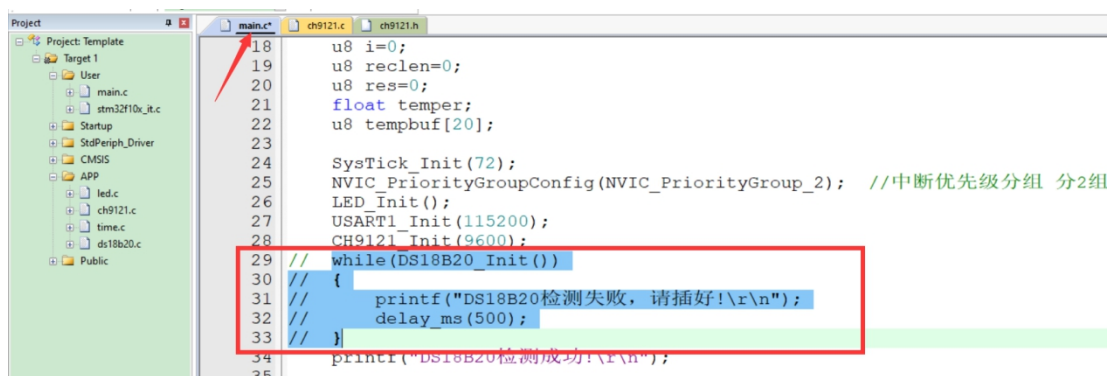


可以看到，开发板已经将 DS18B20 温度传感器采集的温度值上传到网络端。接下来，我们可以在网络发送区发送“LED_ON”字符串即可控制板载 LED 点亮，发送“LED_OFF”字符串即可控制板载 LED 灭。如下所示：



注意：对于没有 DS18B20 传感器的用户，可以在 main.c 文件中，屏蔽对

DS18B20 的检测代码，一样可以进行验证网络控制板载 LED。屏蔽代码如下所示：



```
18 u8 i=0;
19 u8 reclen=0;
20 u8 res=0;
21 float temper;
22 u8 tempbuf[20];
23
24 SysTick_Init(72);
25 NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
26 LED_Init();
27 USART1_Init(115200);
28 CH9121_Init(9600);
29 // while(DS18B20_Init())
30 // {
31 //     printf("DS18B20检测失败, 请插好!\r\n");
32 //     delay_ms(500);
33 // }
34 printf("DS18B20检测成功!\r\n");
35
```

6 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网: www.prechin.cn

普中论坛: www.prechin.net

技术电话: 0755-21509063（转技术）